НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №1
«Розгортання середовища та порівняльний аналіз мовних моделей»

з дисципліни: «Обчислювальні методи»

Варіант 7

Виконав: Студент групи АІ-244
Єкимов Олександр
Олекійович

Перевірив: ас. Баранов О.М.

Одеса – 2026

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Виконано запити до двох моделей через ендпойнт /api/generate. Спочатку – просте фактичне питання, потім – задача на міркування (5 запусків для кожної моделі). Перед серією вимірів зроблено прогрівальний запит.

Таблиця 1 – Фактичне питання «Столиця України? Відповідай одним словом.»

Показник	qwen3:4b	llama3.2:3b
response	Київ	Київ
eval_count	241	4
Час, с	3,12	0,24
Токенів/с	84,93	120,23

Таблиця 2 – Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5 / 5	2 / 5
eval_count	1273,4	185,4
Час, с	16,20	2,31

Типова помилка інструктивної моделі: відповідь «10 грн», отримана без урахування того, що різниця у 100 грн розподіляється між двома доданками.

Відповіді на питання (крок 6)

1. Куди пішли токени міркувальної моделі на простому питанні?

Ці токени пішли на генерацію прихованого ланцюжка внутрішніх міркувань — поле thinking (Chain-of-Thought). Модель не відповідає відразу, а витрачає обчислювальний бюджет на самоперевірку, аналіз мови запиту та відсікання помилкових варіантів, перш ніж видати єдине результуюче слово в поле response.

2. На яких типах задач міркувальні моделі дають перевагу, а де вони невиправдані?

Міркувальні моделі незамінні у складних багатоетапних задачах: математиці, написанні й дебагу коду, формальній логіці та алгоритмічних головоломках, де кожен проміжний крок впливає на фінал. Вони абсолютно невиправдані для простих фактичних довідок, коротких перекладів, класифікації тексту чи розмовних чат-ботів, оскільки створюють зайву затримку (latency) і споживають надмірні ресурси.

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Інструктивні моделі навчаються методом класичного SFT (Supervised Fine-Tuning) та RLHF/DPO за принципом «запит → фінальна відповідь», оптимізуючись під стислість і корисність. Міркувальні моделі тренують із використанням покрокового підкріплення (RL, як-от STaR або методи на кшталт PPO/GRPO), заохочуючи модель розгортати ланцюжки роздумів, знаходити власні помилки та самокоригуватися перед генерацією фіналу.

4. Чи означає більша кількість токенів, що модель «розумніша»?

Ні, більша кількість токенів свідчить лише про інший спосіб інференсу — розгортання простору міркувань («test-time compute»), а не про вищий інтелект базових ваг. Модель із довгим ланцюжком думок може впасти у «перемудрування» (overthinking) або циклічну деградацію, натомість для простого завдання компактна інструктивна модель розв'язує задачу значно ефективніше.

5. Як пояснити різницю у швидкості генерації (токенів/с) між моделями?

Головний фактор — розмір моделі: qwen3:4b має приблизно 4 мільярди параметрів, тоді як llama3.2:3b — близько 3 мільярдів, тому кожен прямий прохід (forward pass) у Qwen вимагає помітно більше обчислень і пропускної здатності пам'яті. Крім того, на швидкість впливають внутрішня архітектура уваги (розмір словника, KV-кеш) та оптимізація квантування моделі під конкретне залізо.

Особливості практичного використання міркувальних та інструктивних моделей

Головна відмінність між цими класами полягає в перерозподілі ресурсів: інструктивні моделі націлені на мінімальну затримку (low latency) та передбачуваність формату, через що вони ідеально підходять для UI/UX-інтерфейсів, API-маршрутизації та класичних помічників. Проте в задачах «з пасткою» (як задача про ручку та зошит) їм бракує робочої пам'яті для покрокового контролю, що призводить до інтуїтивних помилок (40% успіху в тестах).

Міркувальні моделі демонструють 100% точність у складній логіці, фактично обмінюючи час користувача на якість результату. Їх доцільно впроваджувати в асинхронних бекенд-пайплайнах, агентних архітектурах та складній аналітиці, але

для тривіальних запитів їх використання є надлишковим через затримку у 10–15 секунд на відповідь.

Висновок за завданням 1

Експеримент показав, що два показники – швидкість генерації та загальний час відповіді – не можна плутати. Перший характеризує обладнання й розмір моделі, другий – обсяг роботи, яку модель собі призначила. На фактичному питанні режим міркування виявився чистою перевитратою: результат однаковий, витрати відрізняються у 13 разів. На задачі про ручку й зошит ситуація протилежна – саме розписаний хід дозволив уникнути інтуїтивно привабливої, але неправильної відповіді. Практичний висновок: тип моделі треба обирати під тип задачі, а для агента з 8 послідовними викликами різниця складе 129,6 с проти 18,5 с, що робить міркувальну модель непридатною для налагодження.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Таблиця 3 – Вплив system-повідомлення

Модель	System	Довжина відповіді, символів	Мова відповіді	Кількість речень	Інструкцію виконано?
llama3.2:3b	лаконічний асистент	1257	Українська (ламаний суржик)	12	Ні
llama3.2:3b	українською, 2 речення	213	Українська (калька)	2	Так (формально)
qwen3:4b	лаконічний асистент	247	Англійська	3	Частково
qwen3:4b	українською, 2 речення	187	Українська	2	**Так**

Таблиця 4 – Витрати токенів

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент	50	357	407
llama3.2:3b	українською, 2 речення	54	62	116
qwen3:4b	лаконічний асистент	53	3954	4007
qwen3:4b	українською, 2 речення	62	813	875

Відповіді на питання (крок 6)

1.Яка модель точніше дотрималась обмеження «максимум двома реченнями»?

Обидві моделі згенерували рівно по два речення, проте qwen3:4b дотрималась обмеження якісніше, зберігши чіткий технічний зміст без втрати суті.

2.Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?

Пряму вказівку відповісти українською моделі не порушили, проте на першому етапі без явної вказівки мови qwen3:4b відповіла англійською на український запит користувача.

3.Як змінився prompt_tokens при зміні system-повідомлення і чому?

Показник prompt_tokens зріс в обох моделей (з 50 до 54 у Llama та з 53 до 62 у Qwen), оскільки нове системне повідомлення містило довший і складніший текст інструкції.

3.Чому в qwen3:4b значення completion_tokens значно більше, ніж у llama3.2:3b, навіть при однаковій довжині видимої відповіді?

qwen3:4b є міркувальною моделлю, тому API враховує в completion_tokens не лише видимий текст, а й сотні прихованих токенів внутрішнього ланцюжка думок (thinking).

Крок 7. Відкрити будь-який чат-бот (ChatGPT, Gemini, Claude тощо), надіслати йому свої дані з таблиць і поставити п’ять питань

1. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови відповіді?

Різниця зумовлена архітектурою та даними донавчання: інструктивні моделі оптимізовані слідувати шаблонам поверхово і часто збиваються на розпливчастих формулюваннях, а міркувальні моделі здатні проаналізувати обмеження у своєму ланцюжку думок. Крім того, на вибір мови впливає мовний розподіл у навчальній вибірці кожної конкретної моделі.

2. Чим system-повідомлення відрізняється від user-повідомлення з точки зору моделі — чи справді воно має більший пріоритет?

Архітектурно для моделі обидва типи є звичайними токенами у вхідному контексті, розділеними спеціальними системними тегами (im_start system тощо). Хоча під час RLHF-навчання модель привчають надавати системним тегам вищий авторитет, безумовного пріоритету це не гарантує, і останні речення з повідомлення користувача можуть перебити системні настанови.

3. Чому міркувальна модель витрачає більше вихідних токенів, ніж інструктивна, навіть коли видима відповідь однакової довжини?

Перед видачею фінального результату міркувальна модель проводить «обчислення під час інференсу» (test-time compute), генеруючи токени внутрішнього монологу. Сервер Ollama підсумовує ці службові кроки міркування в загальну лічилку completion_tokens, хоча клієнту віддається лише лаконічна відповідь.

4. Що станеться, якщо передати моделі суперечливі інструкції в system і user, — яку вона виконає?

Результат недетермінований і залежить від балансу навчання: частіше перемагає user-повідомлення через ефект свіжості контексту (recency bias), оскільки воно розташоване ближче до генерації. Проте добре відкалібровані моделі можуть або спробувати знайти компроміс, або вказати на внутрішню суперечність.

5. Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?

Малі моделі (3–4 млрд параметрів) мають менший обсяг внутрішніх асоціативних зв'язків і обмежений словник токенізатора для не-англійських мов. Їм не вистачає ємності уваги, щоб одночасно утримувати складні інструкції з формату та контролювати генерацію рідкісними для їхньої вибірки українськими токенами.

Думки щодо ролі system-повідомлення

Надійність керування: system-повідомлення задає загальний контекст і стиль поведінки, проте воно не є гарантованим фільтром або непорушним правилом. Розпливчасті інструкції на кшталт «бути лаконічним» малі моделі зазвичай ігнорують, тоді як суворі чисельні межі («максимум 2 речення») спрацьовують значно стабільніше.

Чи достатньо самої інструкції: Для гарантованого формату (особливо JSON або суворого ліміту слів) одного лише промпту недостатньо. Його необхідно підкріплювати обмеженнями інференсу на рівні рушія (параметри format="json", передача схеми структурованого виводу або керування stop-токенами).

Що робити при ігноруванні інструкції:

Додавати демонстраційні приклади формату прямо в промпт (підхід Few-Shot Prompting).

Переносити критичні обмеження безпосередньо в кінець user-повідомлення.

Знижувати температуру генерації (temperature=0.1–0.2) для більш передбачуваного слідування правилам.

Задіяти Structured Outputs (Grammar-based sampling) в API для апаратного контролю синтаксису.

Висновок за завданням 2

Експеримент підтвердив, що system-повідомлення є дієвим інструментом налаштування контексту, проте надійність його виконання суттєво залежить від чіткості інструкцій та типу моделі. Розпливчасті директиви («лаконічний асистент») призводять до непередбачуваної поведінки: інструктивна модель llama3.2:3b згенерувала надмірно розлогий текст на 12 речень, а міркувальна qwen3:4b відповіла англійською мовою. Перехід до кількісних і мовних обмежень («українською, максимум 2 речення») забезпечив суворе дотримання формату обома моделями, водночас qwen3:4b продемонструвала значно вищу термінологічну точність і якість української мови.

Аналіз витрат токенів (response.usage) показав ключову архітектурну відмінність: у міркувальної моделі qwen3:4b показник completion_tokens виявився у рази більшим (813 та 3954 токени проти 62 та 357 у Llama) навіть за лаконічної фінальної відповіді, оскільки в нього враховуються приховані токени ланцюжка думок (thinking). Практичний висновок для розробки: для гарантованого контролю формату в малих LLM самої інструкції недостатньо — її необхідно підкріплювати точними числовими рамками, зниженням температури генерації

або механізмами структурованого виводу (JSON-схеми, Few-Shot приклади), а вибір моделі повинен враховувати компроміс між вартістю/швидкістю генерації токенів та змістовною точністю відповіді.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Таблиця 5 – Порівняння локальної та хмарної моделі

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	openai/gpt-oss-20b
Час відповіді, с	3.00 с	0.62 с
completion_tokens	63	275 (включно зі 107 reasoning)
Довжина відповіді, символів	219 (без пробілів: 185)	485 (без пробілів: 409)
Мова відповіді	Українська (з артефактами токенізації: «phảnмінь»)	Українська
Фактичних помилок у відповіді	Груба фактологічна галюцинація (поняття ОС сплутано з хімією/фізикою)	1 фактична помилка (у тесті 2_format вказано 1903 рік для першого кінопоказу братів Люм'єр замість 1895 року)

Крок 5. Відповісти на питання:

1.Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Лише 3 рядки конфігурації у словнику провайдера: змінити base_url, підставити api_key замість фіксованого значення та оновити назву model, тоді як сам логічний код виклику API залишився повністю незмінним.

2.Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

Хмарна модель надала 1 фактичну помилку (у тесті 2_format вказано 1903 рік для першого кінопоказу братів Люм'єр замість 1895 року).

3.Чи були в локальної моделі фактичні помилки або проблеми з українською мовою?

Так, зафіксовано як критичну фактичну помилку (галюцинацію про «хімічні phảnмінь»), так і грубі дефекти токенізації: випадкові іноземні гліфи та злипання слів («певномучасі»).

4.Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит летить через інтернет?

Хмарний провайдер Groq використовує спеціалізовані надшвидкі чипи LPU (Language Processing Units) з високою пропускною здатністю пам'яті (SRAM), швидкість генерації яких багаторазово перекриває затримку мережевого пінгу порівняно зі звичайним споживчим залізом (CPU/GPU).

Крок 6. Питання до чат-бота:

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?

OpenAI першими масово випустили простий і зручний REST/JSON протокол (ендпоінт /v1/chat/completions), під який спільнота написала сотні SDK, бібліотек та готових фреймворків (LangChain, LlamaIndex). Підтримка цього інтерфейсу іншими провайдерами та Ollama дозволяє розробникам змінювати рушії чи бекенди заміною лише кількох рядків конфігурації (base_url та model) без переписування кодової бази.

2. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?

Розмір моделі є фундаментальним для місткості знань: більша кількість параметрів (20B–70B) забезпечує глибоке розуміння контексту, логіку міркувань і багатий словниковий запас для не-англійських мов. Проте якість даних попереднього навчання (pre-training data), донавчання на інструкціях (RLHF/SFT) та точність складеного системного промпту можуть нівелювати перевагу за розміром для вузьких спеціалізованих завдань.

3. Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?

Швидкість генерації тексту (time-per-output-token) у LLM обмежена не стільки обчисленнями, скільки швидкістю зчитування ваг із пам'яті (memory bandwidth). Сервери інференсу в хмарі розгортають моделі на датацентрових кластерах із пам'яттю HBM3 або ультрашвидких LPU (як у Groq), які видають 200–500 токенів/с, тоді як локальний інференс на споживчому ПК стикається з вузьким горлечком шини оперативної пам'яті DDR4/DDR5 чи звичайної VRAM.

4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

Головні загрози — це витік конфіденційних даних клієнтів (PII), порушення регламентів (GDPR, HIPAA, банківська таємниця) та потенційне використання вхідних запитів для донавчання публічних моделей. У продакшені компанії укладають корпоративні угоди з гарантією Zero Data Retention (ZDR), маскують персональні дані перед відправкою (анонімізація) або розгортають ізольовані інстанси у виділених хмарах (Azure OpenAI / AWS Bedrock).

5. Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

Rate limit — це програмне обмеження кількості запитів (RPM — requests per minute) та токенів (TPM — tokens per minute), яке сервіси впроваджують для запобігання перевантаженню інфраструктури та зловживанням. Розробники обходять ці бар'єри впровадженням черг і повторних спроб з експоненційним відкладенням (exponential backoff з jitter), кешуванням типових відповідей (Redis), балансуванням запитів між декількома ключами/провайдерами (fallback routing) або переходом на платний tier.

Власні думки: вибір між локальною та хмарною моделлю

Коли обирати локальну модель:

Сувора конфіденційність: Робота з комерційною таємницею, медичними картками, вихідним кодом закритих проєктів чи персональними даними, які заборонено передавати третім сторонам.

Автономність та стабільність: Системи без доступу до інтернету (edge-пристрої, польові умови, закриті контури безпеки) або за потреби виключити залежність від сторонніх сервісів, їхніх збоїв та підвищення тарифів.

Вузькі завдання: Класифікація тексту, просте парсування логів, базове вилучення сутностей, де маленької моделі (3B–8B) достатньо після донавчання (LoRA).

Коли обирати хмарну модель:

Високі вимоги до логіки та якості мови: Складна аналітика, генерація юридичних/технічних текстів, багатокрокове програмування, де малі моделі дають збій або галюцинують.

Висока швидкість та масштабованість: Продукти з великою кількістю користувачів, де потрібна генерація сотень токенів за секунди без купівлі власного дорогого серверного обладнання.

Швидкий запуск (Time-to-market): Можливість одразу підключитися через OpenAI-сумісне API до флагманських моделей без налаштування драйверів, квантування та адміністрування серверів.

Висновок за завданням 3

Порівняльний аналіз локального інференсу (llama3.2:3b через Ollama) та хмарного сервісу (openai/gpt-oss-20b через Groq Cloud) продемонстрував суттєву різницю в продуктивності, якості генерації та інженерних підходах до інтеграції LLM.

Уніфікація інтерфейсу та простота міграції:

Використання OpenAI-сумісного API є ключовим індустріальним стандартом. Завдяки абстракції у файлі providers.py перехід від локального запуску до хмарного провайдера не вимагав переписування бізнес-логіки скрипту — достатньо було змінити лише параметри конфігурації (base_url, api_key, model), що підтверджує високу гнучкість сучасної екосистеми бібліотек для роботи з LLM.

Швидкість генерації та апаратні обмеження:

Незважаючи на необхідність передачі даних через інтернет, хмарна модель відпрацювала майже в 5 разів швидше за локальну (0.62 с проти 3.00 с). Це обумовлено використанням датацентрових апаратних прискорювачів Groq (LPU) з надвисокою пропускною здатністю пам'яті (SRAM), тоді як локальний інференс на звичайному споживчому залізі обмежений низькою пропускною здатністю шини системної оперативної пам'яті.

Якість знань та мовна стійкість:

Експеримент яскраво виявив межі спроможності надмалих моделей (3 млрд параметрів). Локальна llama3.2:3b продемонструвала повну фактологічну галюцинацію, сплутавши комп'ютерні терміни з хімічними процесами, та допустила грубі артефакти токенізації в українській мові. Натомість більша

модель openai/gpt-oss-20b надала технічно бездоганне, академічно точне та чітко структуроване пояснення операційних систем, використавши внутрішні міркування (reasoning_tokens=107).

Інженерний компроміс (Trade-off):

Вибір архітектури рішення визначається балансом між безпекою та якістю. Локальні моделі залишаються безальтернативними для завдань із критичними вимогами до конфіденційності (Zero Data Leakage), відсутності підключення до мережі та нульових витрат на API-трафік. Хмарні ж моделі є необхідними для складних аналітичних задач, багатокрокових міркувань і високих навантажень, де потрібна швидка генерація сотень токенів за секунду без розгортання власної дорогої серверної інфраструктури.

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Таблиця 7 – Результати вимірювання швидкості

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	2.079	2.909	83.1
локальна	llama3.2:3b	середній	2.067	3.380	120.5
локальна	llama3.2:3b	довгий	2.103	7.487	227.4
хмарна	openai/gpt-oss-20b	короткий	0.248	0.259	48.9
хмарна	openai/gpt-oss-20b	середній	0.388	0.497	678.0
хмарна	openai/gpt-oss-20b	довгий	0.535	1.465	1761.8

Таблиця 8 – Холодний і теплий старт локальної моделі

Показник	Холодний старт	Теплий старт
TTFT, с	3.850	2.079
Всього, с	4.680	2.909

Крок 5. Побудувати стовпчикову діаграму (matplotlib):



Крок 6. Відповіді на питання

1.У якої моделі кращий TTFT і чому саме в неї?

Кращий (значно менший) TTFT продемонструвала хмарна модель (0.25–0.54 с проти ~2.07 с у локальної), оскільки датацентрові прискорювачі Groq (LPU) опрацьовують вхідний промпт на порядки швидше, перекриваючи навіть мережеву затримку.

2.Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні — поясніть.

На довгих і середніх промптах так, проте на короткому запиті локальна модель формально показала більше символів/с (83.1 проти 48.9), оскільки хмарна модель видала лаконічну відповідь всього з кількох слів, де мережевий оверхед викривив питомий розрахунок швидкості.

3.Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промпту до довгого?

На короткому запиті локальна модель видавалася конкурентною за питомою швидкістю, проте на довгому промпті хмарна модель відірвалася майже у 8 разів (1761.8 симв./с проти 227.4 симв./с), розкривши максимальну пропускну здатність пам'яті.

4.Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Повтори відрізнялися в межах 5–15% через коливання завантаження процесора ПК та мережевий jitter інтернет-каналу. Одного запуску недостатньо, адже випадковий стрибок пінгу чи фоновий процес операційної системи спотворили б об'єктивну оцінку.

5.Чому холодний старт повільніший за теплий?

При холодному старті операційна система повинна зчитати кілька гігабайтів ваг нейромережі з накопичувача (SSD/HDD) у RAM/VRAM, тоді як при теплому старті модель уже завантажена в пам'ять і готова до обчислень.

Крок 7. Відповіді на 5 питань

1. Що фізично відбувається під час TTFT — часу до першого токена — і чому цей показник вимірюють окремо від загальної швидкості генерації?

Під час TTFT виконується передача запиту мережею, токенізація тексту та фаза попереднього заповнення (prefill), коли модель паралельно прораховує матричні множення для всього вхідного промпту й формує початковий KV-кеш. Цей показник вимірюють окремо, тому що він відображає затримку реакції інтерфейсу (чуйність для користувача) перед початком посимвольного потокового виводу.

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?

Гірший TTFT у хмарі зазвичай виникає через мережевий пінг (RTT), маршрутизацію SSL-з'єднання та очікування в черзі запитів сервісу (queue time). Проте фаза генерації токенів спирається на спеціалізовані датацентрові чипи (LPU/H100) з надширокою шиною пам'яті (HBM/SRAM), що забезпечує значно вищу швидкість потокового виводу, ніж споживча пам'ять DDR4/DDR5 ПК.

3. Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?

Холодний старт — це запуск інференсу за відсутності моделі в оперативній пам'яті, коли Ollama змушена виконувати блокуюче дискове зчитування файлу ваг (вагою 2–8 ГБ) та виділяти адресний простір. Наступні (теплі) запуски не витрачають час на дисковий I/O, оскільки вся структура тензорів і граф обчислень уже постійно утримуються в RAM або VRAM відеокарти.

4. Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?

Одиничний вимір є вразливим до випадкових шумів: сплесків мережевої затримки, роботи збирача сміття (Garbage Collector), оновлень ОС чи фонових потоків процесора. Медіана, на відміну від середнього арифметичного, відсікає аномальні викиди (outliers) і відображає типову реальну поведінку системи.

5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Обробка вхідного промпту (prompt prefill) є операцією, обмеженою обчисленнями (compute-bound), і всі його токени обробляються паралельно за один прохід через ядро матричного множення GPU. Натомість генерація відповіді авторегресивна й покрокова: кожен наступний токен генерується окремим проходом із повторним зчитуванням усіх гігабайтів ваг із пам'яті, що є операцією, суворо обмеженою пропускною здатністю пам'яті (memory-bandwidth bound).

Власні думки: важливість показників для різних сценаріїв

Для інтерактивного чат-бота найважливішим показником є TTFT (Time To First Token):

У живому інтерфейсі з увімкненим стрімінгом користувач оцінює чуйність системи саме за тим, як швидко з'явилося перше слово. Якщо TTFT становить 0.3 секунди, людина сприймає інтерфейс миттєвим і починає читати текст у міру його виводу, навіть якщо загальна швидкість генерації буде середньою. Навпаки, довгий TTFT (наприклад, 4–5 секунд очікування перед першим символом) створює відчуття зависання програми, навіть якщо потім весь текст висиплеться за частку секунди.

Для фонової пакетної обробки (Batch Processing) великих обсягів тексту критичним є загальний час виконання (Total Time) та пропускна здатність (симв./с або tokens/sec):

Під час сумаризації тисяч документів, класифікації транзакцій, аналізу логів або міграції баз даних перед екраном немає людини, яка чекає на перший токен. Тут затримка запуску ролі не грає, а ключовими факторами стають сумарна пропускна

здатність системи (Throughput), вартість токена та швидкість обробки всієї черги завдань за одиницю часу.

Висновок за завданням 4

У ході виконання завдання було проведено комплексне бенчмаркінг-дослідження часових та швидкісних метрик локальної моделі (llama3.2:3b через Ollama) та хмарної моделі (openai/gpt-oss-20b через Groq Cloud) в режимі потокової генерації (stream=True) з використанням трьох типів вхідних промптів (короткий, середній, довгий).

Динаміка часу до першого токена (TTFT):

Хмарна модель продемонструвала значно кращу затримку відгуку: її показник TTFT склав від 0.25 до 0.54 с, що в 4–8 разів швидше за локальну модель (~2.07–2.10 с). Це підтверджує, що стадія попередньої обробки промпту (prompt prefill) на спеціалізованих апаратних прискорювачах Groq (LPU) виконується практично миттєво, повністю нівелюючи мережеві затримки передачі пакетів через інтернет.

Пропускна здатність та масштабованість генерації:

Швидкість покрокової генерації токенів кардинально різниться залежно від довжини тексту. Якщо на короткому запиті локальна модель показала конкурентну швидкість у символах за секунду через лаконічність відповіді, то на довгому промпті перевага хмарної моделі стала абсолютною — 1761.8 симв./с проти 227.4 симв./с у локальної. Це обумовлено високою пропускною здатністю внутрішньої пам'яті хмарних серверів, тоді як локальний інференс суворо обмежується пропускною здатністю шини системної оперативної пам'яті (DDR4/DDR5).

Вплив кешування (Холодний vs Теплий старт):

Експеримент із зупинкою моделі (ollama stop) наочно показав природу холодного старту: перший виклик супроводжується значною затримкою (близько 3.85 с до першого токена), викликаною необхідністю зчитування гігабайтів ваг із дискового накопичувача у виділений простір RAM/VRAM. Наступні «теплі» виклики стабілізують TTFT на рівні ~2.08 с, оскільки ваги моделі та граф обчислень залишаються закешованими в пам'яті процесу.

Методологічна оцінка метрик:

Використання медіани за трьома вимірами виявилося критично необхідним для усунення випадкових шумів — коливань навантаження ОС та мережевого джитера. Результати експерименту чітко розмежовують сфери застосування метрик: TTFT є визначальним фактором комфорту користувача в інтерактивних

чат-інтерфейсах, тоді як швидкість генерації (симв./с або tokens/sec) і загальний час виконання є вирішальними для пакетної та фонової обробки великих масивів тексту.

Завдання 5. Оцінювання якості моделей за рубрикою

Таблиця 13 –Оцінка якості за рубрикою

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	0	2	2
2. Формат	0	1	1
3. Стиснення	0	1	2
4. Код	2	2	1
5. Логіка	0	2	2
6. Українська	0	2	2
Разом /12	2 / 12	10 / 12	10 / 12

Таблиця 14 – Розбіжність оцінок між двома прогонами (так / ні)

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	Ні	Ні	Ні
2. Формат	Ні	Ні	Ні
3. Стиснення	Ні	Ні	Ні
4. Код	Ні	Ні	Ні
5. Логіка	Ні	Ні	Ні
6. Українська	Ні	ні	Ні

Крок 6. Відповіді на питання

1.Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?

Найбільше балів розділили qwen3:4b та хмарна openai/gpt-oss-20b (по 11/12), а основний відрив від llama3.2:3b сформувався на логічній задачі, валідації JSON, фактологічній точності та якості української мови.

2.На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

Бали за шкалою критеріїв не розійшлися в жодному завданні (оцінки були стабільними 0 або 2), що свідчить про високу повторюваність рівня спроможності моделей, попри текстові варіації через ненульову температуру.

3.Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Так, хмарна модель openai/gpt-oss-20b в обох прогонах повернула абсолютно чистий синтаксичний JSON-масив без зворотних лапок (```), тоді як локальна llama3.2:3b не змогла побудувати коректний JSON взагалі.

4.Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?

У моделі llama3.2:3b виявлено критичну деградацію мови: суміш англійських та польських слів («purposes», «frequently employs», «narracyjної»), русизми («шаг за шагом», «окончова») та артефакти символів Unicode, тоді як у openai/gpt-oss-20b мова була граматично бездоганною.

Крок 7. Обов’язковий пункт – приклад провалу. Обрати один найпоказовіший випадок і навести

Модель: llama3.2:3b (Локальна)

Завдання: 5_logic (Багатокрокова арифметична задача)

Повний текст запиту:

Зал 180 місць, сеанс 1 год 45 хв + 15 хв перерва, робота з 10:00 до 23:00. Скільки сеансів і глядачів? Покажи хід розв'язання.

Повний текст відповіді (Спроба 1, без скорочень):

Давайте спробуємо розв'язати проблему шаг за шагом:

Високий zaal з 180 місцем.

Сеанс триває 1 год 45 хв + 15 хв перерва.

Робота з 10:00 до 23:00.

Похід:

Високий зал з 180 місцем означає, що зал встигає весь зал за одну сеанс.

Сеанс триває 1 год 45 хв + 15 хв перерва, що означає, що сеанс триває 1 год 60 хв.

Робота з 10:00 до 23:00 означає, що робочий час займає 13 хв (з 10:00 до 23:00, 13 хв).

Поскольку сеанс триває 1 год 60 хв, то на сеансі може присутствовать не більше 13 хв (високий зал з 180 місцем).

Знаємо, що зал встигає весь зал за одну сеанс, що означає,що на ceANCEмогуто присутствовать не більше 180 глядачів.

Поскольку на сеансі можnette присутствовать не більше 13 хв, а глядачів не більше 180, то на сеансі могуть присутствовать не більше 180/13 = 13,615 глядачів (приблизно).

Тепер, щоб визначити кількість сеансів, нам потрібно оцінити, яком慶 багато сеансів можуть відбіти протягом робочого часу.

Робочий час займає 13 хв, а сеанс триває 1 год 60 хв, що означає,що на робочому місці може відбіти не більше 13/60 = 0,217 сеансів (приблизно).

Поскольку сеансів не можна маєти частин, то на робочому місці можуть відбіти не більше 1 сеанс (приблизно).

Тепер, щоб визначити кількість глядачів, нам потрібно визначити, якомньо багатьох глядачів можуть присутствувати на сеансах протягом робочого часу.

Робочий час займає 13 хв, а сеанс триває 1 год 60 хв, що означає, що на сеансах можуть відбіти не більше 13 сеансів (приблизно).

Поскольку на сеансах можуть відбіти не більше 1 сеанс на робочому місці, то на робочому місці можуть відбіти не більше 13 сеансів (приблизно).

Поскольку на сеансах можуть відбіти не більше 13 сеансів, то на сеансах можуть присутствувати не більше 13x180 = 2340 глядачів (приблизно).

Окончова відповідь: На сеансах можуть відбіти 1 сеанс, а глядачів не більше 2340.

Яка правильна відповідь мала бути:

Загальний робочий час: з 10:00 до 23:00 = 13 годин = 780 хвилин.

Тривалість повного циклу (сеанс + перерва): 1 год 45 хв + 15 хв = 105 хв + 15 хв = 120 хвилин (2 години).

Кількість повних сеансів: 780 / 120 = 6.5 \ повних сеансів (залишок 60 хв недостатній для фільму).

Максимальна кількість глядачів: 6 \ 180 = 1080 глядачів.

Пояснення, чому так сталося:

Провал викликаний архітектурним обмеженням розміру моделі (3 млрд параметрів) та слабким механізмом внутрішнього ланцюжка міркувань (Chain-of-Thought):

Втрата одиниць вимірювання (Dimensionality Collapse): Модель порахувала різницю 23 - 10 = 13, але підмінила години хвилинами («робочий час займає 13 хв»).

Відсутність валідації суперечностей: Модель одночасно стверджує, що відбудеться «1 сеанс», але в кінці множить 13 180 = 2340 глядачів, демонструючи повну втрату контексту попередніх кроків.

Мовна деградація при математичному навантаженні: Під час обчислень увага трансформера (attention) перевантажилася числовими зв'язками, через що токенізатор почав продукувати суржик («поскольку», «могуто»), французькі частки («можnette») та випадкові ієрогліфи («яком麼»).

Крок 8. Надіслати будь-якому чат-боту (ChatGPT, Gemini, Claude тощо) свою підсумкову таблицю балів і поставити п’ять питань

1. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

В індустрії застосовують автоматизовані синтетичні бенчмарки (MMLU для ерудиції, GSM8K для математики, HumanEval для коду), які тестують моделі на тисячах стандартизованих запитань із точним метричним підрахунком accuracy. Також широко використовують підхід «LLM-as-a-Judge» (коли сильніша модель, наприклад GPT-4, оцінює відповіді менших за критеріями зв'язності та фактів) та сліпе порівняння людиною на платформах на кшталт LMSYS Chatbot Arena.

2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?

Під час фази вирівнювання (RLHF та instruction fine-tuning) моделі навчали на сотнях тисяч прикладів, де будь-який синтаксичний код чи структуровані дані обов'язково загорталися в блоки markdown для коректного рендерингу в UI. Цей патерн настільки глибоко вшитий у ваги трансформера як системна звичка, що для його придушення потрібні спеціальні методи інференсу (наприклад, JSON mode, constrained decoding або граматики BNF).

3. Чому малі мовні моделі гірше пишуть українською, ніж англійською?

Малі моделі (3–4 млрд параметрів) мають обмежену місткість словника токенізатора і навчалися на корпусах, де український сегмент складав менше 0.5–1% від усього обсягу даних. Через це українські слова дробляться на дрібні суб-токени (байт-пари), що вимагає більшої кількості кроків трансформера для генерації одного слова й призводить до змішування мовних шарів, галюцинацій та суржику.

4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?

Причиною є стохастичний вибір наступного токена, який регулюється параметром temperature (у вашому експерименті 0.7) та алгоритмами семплювання (top_p). Ненульова температура змушує модель вибирати не найбільш імовірне слово (як при greedy decoding з temperature=0), а випадкове зі зваженого розподілу ймовірностей, що в одному випадку скеровує генерацію на правильний логічний ланцюг, а в іншому — на галюцинацію.

5. Що таке галюцинація мовної моделі і чому модель радше вигадає факт, ніж визнає незнання?

Галюцинація — це генерація переконливо звучного, граматично коректного, але фактологічно хибного тексту через статистичний характер роботи архітектури трансформера, яка прогнозує найбільш ймовірні токени, а не перевіряє онтологічні знання. Модель схильна вигадувати відповідь через специфіку функції втрат під час навчання: її штрафували за порожні чи ухильні відповіді й заохочували за завершення тексту відповідно до форми промпту.

Власні думки: аналіз методики та вибір моделі

Надійність оцінки за рубрикою: Рубрика виявилася напрочуд об'єктивним інструментом, оскільки чітко розкрила функціональні межі архітектур: вона одразу відокремила «декоративну» якість поверхневого діалогу від здатності моделі слідувати жорстким бізнес-правилам (JSON, відбір параметрів).

Найпоказовіші завдання: Найбільш інформативними виявилися Завдання 2 (формат) та Завдання 5 (логіка). Саме на них компактна модель остаточно провалилася, продемонструвавши, що без глибокого апарату міркувань (CoT) генерація математики та валідних структур перетворюється на беззмістовне комбінування схожих слів.

Вибір моделі для українськомовного тексту: Для будь-якої практичної роботи з текстами українською мовою вибір однозначно падає на хмарну модель (openai/gpt-oss-20b) або на спеціалізовані моделі з якісним CoT (як qwen3:4b). Мала llama3.2:3b категорично непридатна для українського контенту через

агресивне просочування суржику, деградацію лексики та пошкодження діакритики.

Висновок за завданням 5

Дослідження якісних характеристик генерації за стандартизованою рубрикою (факти, суворе дотримання формату, стиснення зі збереженням числових сутностей, програмування, багатоетапна логіка та робота з українською мовою) виявило фундаментальну прірву між надмалими загальними моделями та моделями з розвиненим апаратом міркувань або більшим обсягом параметрів.

Критичні обмеження надмалих архітектур (llama3.2:3b — 2/12 балів):

Модель продемонструвала повну неспроможність у завданнях, що вимагають точного вилучення знань та покрокової формальної логіки. Вона допустила грубі хронологічні галюцинації (помилка у даті першого кінопоказу на десятки років), катастрофічно провалила арифметичну задачу (підміна годин на хвилини та втрата цілісності умов) і зламала синтаксис JSON, унеможлививши програмний парсинг через json.loads(). Крім того, виявлено глибоку деградацію української локалізації: змішування англійських слів, суржику та пошкоджених символів Unicode. Єдиним успішним кейсом виявилася генерація простої Python-функції, що пояснюється надмірною представленістю коду в датасетах попереднього навчання.

Ефективність міркувальних та масштабних моделей (qwen3:4b — 11/12 балів та openai/gpt-oss-20b — 10/12 балів):

Як компактна міркувальна модель qwen3:4b, так і хмарна openai/gpt-oss-20b впевнено впоралися з виділенням ключових числових метрик при стисненні та бездоганним покроковим розв'язанням логічної задачі (вірно визначено 6 сеансів та 1080 глядачів). Хмарна модель продемонструвала багату літературну стилістику української мови та ідеальне слідування негативним інструкціям («без markdown, лише JSON»), повернувши синтаксично чистий масив, готовий до прямої десеріалізації. Проте у завданні 2_format вона припустилася точкової хронологічної галюцинації, вказавши рік першого кінопоказу братів Люм'єр як 1903 замість 1895 року, через що її загальний бал було скориговано до 10/12.

Стійкість та детермінованість:

Аналіз двох прогонів при температурі temperature=0.7 показав абсолютну стабільність балів за рубрикою: попри варіації у формулюваннях і стилістиці речень, рівень спроможності кожної моделі залишився незмінним. Це підтверджує надійність запропонованої методики оцінювання — якісні дефекти

(або їх відсутність) є системними властивостями ваг і внутрішнього ланцюжка думок моделі, а не випадковими поодинокими збоями.

Практична рекомендація для розробки:

Експеримент довів, що компактні моделі розміром ~3 млрд параметрів без вбудованого CoT (Chain-of-Thought) категорично не підходять для ролей API-агентів (структурований вивід даних), бізнес-аналітики чи генерації контенту українською мовою. Для промислових україномовних систем та задач з автоматизованою обробкою структур даних вибір має зупинятися на більших хмарних рішеннях або спеціалізованих reasoning-моделях з обов'язковим примусовим синтаксичним контролем виводу (JSON Schema / Constrained Decoding).

Завдання 6. Керування випадковістю: temperature і top_p

Таблиця 16 –Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче	Правильних із 5 (фактичне
temperature	0.0	2	0
temperature	0.7	5	-
temperature	1.5	5	0
top_p	0.1	1	-
top_p	0.5	5	-
top_p	1.0	5	-

Таблиця 17 – Перевірка відтворюваності

Показник	Значення
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1
Усі відповіді однакові (так / ні)	так

Крок 7. Відповіді на питання

1.Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

Для агента слід встановити temperature=0.0 (або максимум 0.1–0.2), оскільки це мінімізує ентропію і змушує модель суворо дотримуватися синтаксичних правил та заданої схеми без самовільних відхилень.

2.Яке — генератору варіантів заголовків?

Для генератора заголовків оптимальним є діапазон temperature=0.7–0.9, який забезпечує лексичне різноманіття, креативність та незаїжджені асоціації без скочування в галюцинації.

3.Чим вплив top_p відрізняється від впливу temperature за характером, а не за силою?

temperature плавно згладжує або загострює весь розподіл імовірностей усього словника (softmax), тоді як top_p динамічно відсікає довгий хвіст малоймовірних токенів, залишаючи лише компактне ядро найбільш підходящих кандидатів.

4.Чи означає temperature=0 повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?

Теоретично так, і експеримент показав результат True (10 з 10 однакових викликів), проте на практиці в розподілених системах можливий незначний недетермінізм через паралелізм обчислень з плаваючою комою на GPU.

5.Наведіть найбезглуздішу відповідь, яку модель видала при temperature=1.5, і поясніть, чому вона взагалі могла з’явитися.

Відповідь: «Рік пер◌ॆшого кіпопасу братів Люм ’ер, або перший прикорю Бранжєра було 11 років». При температурі 1.5 модель штучно підняла шанси токенів із «хвоста» ймовірностей, через що токенізатор почав вибирати випадкові символи індійського письма деванагарі (◌ॆш), викривляти слова («кіпопасу») та синтезувати абсурдні факти.

Крок 8. Надіслати будь-якому чат-боту (ChatGPT, Gemini, Claude тощо) свої дані з таблиць і поставити п’ять питань

1. Що саме математично робить параметр temperature із розподілом ймовірностей наступного токена?

Параметр ділить логіти (сирі виходи моделі перед функцією softmax) на значення T: $P_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$. При $T \to 0$ При T to 0 різниця між логітами штучно зростає до нескінченності, перетворюючи розподіл на функцію argmax (жадібний вибір єдиного піка), а при T > 1 логіти зрівнюються, роблячи розподіл плоским і майже рівномірним по всьому словнику.

2. Чим top_p відрізняється від temperature — обидва впливають на різноманітність, але за яким механізмом?

temperature трансформує сам профіль кривої ймовірностей для абсолютно всіх слів у словнику, змінюючи контраст між ними. top_p (ядерне семплювання / nucleus sampling) сортує токени за спаданням і відсікає все, що виходить за межі накопичувальної суми ймовірностей p, тобто динамічно змінює розмір пулу доступних кандидатів залежно від впевненості моделі в поточному контексті.

3. Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?

Більшість дослідників та офіційна документація (зокрема OpenAI) рекомендують змінювати лише один із них, фіксуючи інший на нейтральному значенні (наприклад, тримати top_p=1.0 при тюнінгу temperature). Їхня спільна модифікація створює складну інтерференцію, коли один фільтр згладжує хвіст, а інший раптово його обрізає, що ускладнює прогнозування та відтворення поведінки системи.

4. Чому навіть при temperature=0 мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?

На хмарних та багатопотокових відеокартах операції множення матриць розпаралелюються між тисячами ядер CUDA/Tensor Cores. Через неасоціативність додавання чисел із рухомою комою (floating-point precision: (a + b) + c \ a + (b + c) на рівні мікроархітектури) зміна порядку накопичення потоків може змінити останній біт логіта, що при рівних шансах призводить до вибору іншого найближчого токена.

5. Які значення temperature використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?

Для екстракції сутностей, генерації строгого JSON, парсингу або кодогенерації використовують значення в межах 0.0 – 0.2 задля стабільності та дотримання схеми. Для творчих завдань (копірайтинг, брейнштормінг, художні тексти) стандартом є 0.7 – 0.9, де модель демонструє гнучкість асоціацій без ризику граматичної деградації.

Власні думки: вибір параметрів для наступних робіт

Вибір параметрів для структурованих даних: Для наступних лабораторних робіт, де від моделі вимагається повернення валідного JSON або структурованих записів, я оберу temperature=0.0 та top_p=1.0. Це гарантує максимальну детермінованість кодових конструкцій, закриття лапок і дужок та виключає випадкові синтаксичні відхилення.

Чому налаштування «за замовчуванням» є непридатним: Значення за замовчуванням у більшості LLM-бібліотек становить temperature=0.7 або 1.0.

Такий рівень стохастичності розрахований на розмовний чат, але є критично непридатним для бекенд-інтеграцій: за таких параметрів модель рано чи пізно додасть до JSON коментар від себе, пропустить кому або спаплюжить назву поля, що неминуче призведе до аварійного завершення програми на етапі json.loads().

Висновок за завданням 6

У процесі виконання завдання було експериментально досліджено механізми вибірки токенів (sampling) та вплив гіперпараметрів temperature і top_p на детермінованість, різноманітність і якість генерації відповідей локальної моделі (llama3.2:3b).

Вплив параметра temperature на характер тексту:

Експеримент підтвердив пряму залежність між температурою та ентропією розподілу ймовірностей:

Низька температура (temperature=0.0): активує режим жадібного вибору (greedy decoding), забезпечуючи максимальну повторюваність виводу (100% однакових відповідей у серії з 10 прогонів). Проте цей режим фіксує не лише формулювання, але й внутрішні фактологічні помилки моделі (п'ять із п'яти разів було впевнено повторено хибний рік «1997»).

Помірна температура (temperature=0.7): забезпечує збалансовану генерацію з високим рівнем лексичної варіативності (5 унікальних відповідей із 5) без порушення граматичної цілісності мови.

Екстремальна температура (temperature=1.5): призводить до розмивання піків розподілу softmax і катастрофічної деградації тексту. Модель починає обирати малоймовірні суб-токени з «хвоста», що спричиняє появу псевдослів, пошкодження логіки та артефактів токенізації (вкраплення деванагарі, ієрогліфів і змішування мов).

Порівняння механізмів temperature та top_p:

Досліджено принципову різницю в природі фільтрації: якщо temperature діє аналогово, плавно згладжуючи весь профіль логітів словника, то top_p (ядерне семплювання) працює як динамічний поріг відсікання. При top_p=0.1 модель штучно ізолює лише найвищий пік ймовірності (1 унікальна відповідь із 5), діючи аналогічно жадібному вибору, тоді як збільшення p розширює коло кандидатів без ризику вибору екстремально малоймовірних токенів, як це трапляється при T > 1.0.

Практичне значення для інженерії застосунків:

Отримані дані мають ключове значення для налаштування взаємодії з мовними моделями в програмних системах:

Дефолтні налаштування (temperature=0.7–1.0) категорично непридатні для задач структурованої обробки інформації, оскільки вбудована стохастичність періодично провокує синтаксичні відхилення або порушення структури JSON.

Для API-агентів, парсерів та генерації строгого JSON єдино правильним вибором є встановлення **`temperature=0.0`** (або жорсткого `top_p`), що гарантує валідність синтаксису й стабільність десеріалізації.

Творчі сценарії (неймінг, копірайтинг, брейнштормінг) вимагають значень у діапазоні **0.7–0.9**, де модель розкриває семантичну гнучкість без скочування в галюциногенне марення.

Крок 8. Звести все разом і зробити висновок

Таблиця 18 – Підсумкове порівняння моделей

Критерій	Локальна мала (3B)	Локальна більша (4B/8B)	Хмарна
Швидкість генерації	Помірна (80–230 симв./с на споживчому залізі; суворо впирається в пропускну здатність RAM/VRAM)	Середня (60–150 симв./с на середньому залізі, залежить від обсягу відеопам'яті та квантування Q4_K_M)	Екстремально висока (до 1500–1800 симв./с завдяки датацентровим прискорювачам LPU/H100)
TTFT	Повільний (~2.0–2.1 с при теплому старті, до 3.8–4.5 с при холодному через дисковий I/O)	Помірний/довгий (~1.5–3.5 с залежно від розміру моделі, фази prompt prefill та наявності дискретної GPU)	Мінімальний (~0.25–0.54 с; паралельний prefill на LPU перекриває навіть мережевий пінг)
Якість, балів /12	Низька (2 / 12); схильна до галюцинацій, не тримає логіку, деградує в українській мові	Висока (10–11 / 12); наявний розвинений CoT, впевнено розв'язує логічні та мовні завдання	Висока (11 / 12); бездоганна мовна стилістика, точні факти, глибоке дотримання правил
Стабільність формату (завдання	Критично нестабільна	Задовільна / добра (часто потребує	Відмінна (ідеально формує

2)	(руйнує JSON-синтаксис, ділить рядки крапкою з комою, збоїть парсинг json.loads())	контролю temperature=0.0 або придушення блоків ```json)	синтаксичний JSON-масив без зайвого тексту та markdown-лапок)
Потрібні ресурси	Мінімальні (2–4 ГБ RAM/VRAM, запуск можливий практично на будь-якому офісному ПК/CPU)	Помірні / відчутні (6–10 ГБ VRAM для комфортної швидкості або 16 ГБ швидкої RAM при роботі на CPU)	Нульові вимоги до локального заліза (потрібен лише стабільний інтернет-канал та HTTP-клієнт)
Вартість	Безкоштовно (оплачується лише електроенергія ПК)	Безкоштовно (одноразова вартість відеокарти з достатнім пулом VRAM)	Оплата за використання (pay-as-you-go за токени; наявні безкоштовні ліміти/free-tier у Groq)
Куди йдуть ваші дані	Залишаються суворо на локальній машині (не виходять за межі localhost)	Залишаються суворо на локальній машині (повна автономність і конфіденційність)	Передаються через інтернет на сторонні сервери провайдера (ризик витоку, аналіз провайдером)
Працює без інтернету	Так, на 100% автономна	Так, на 100% автономна	Ні, будь-який обрив зв'язку або блокування API повністю зупиняє систему
Коли обирати саме її	Обирається виключно за крайньої потреби в абсолютній автономності (air-gapped контури, робота в польових умовах без зв'язку) або жорстких вимог до приватності, коли на слабкому залізі потрібно	Обирається для корпоративних, медичних, фінансових або чутливих локальних проектів із суворими вимогами NDA/GDPR, де дані категорично заборонено надсилати в хмару,	Обирається як основний робочий інструмент для інтерактивних користувацьких інтерфейсів (чат-боти), складних бізнес-пайплайнів, вебсервісів і швидкої обробки великих текстів, де пріоритетом є мінімальний час

	виконувати примітивні рутинні задачі: фільтрацію спаму, просту класифікацію, витяг очевидних ключових слів чи шаблонізацію англомовних коротких фраз. Якщо потрібна математика, structured JSON чи якісна українська мова — модель не підходить.	але завдання вимагають складних міркувань (CoT), перевірки коду, математичної точності та зв'язного українського тексту. Вимагає наявності адекватного апаратного забезпечення (GPU від 8 ГБ VRAM).	відгуку (TTFT < 0.5 с), висока пропускна здатність (>1500 симв./с), багатство мови та гарантія валідного JSON, а оброблювані дані є відкритими або захищені стандартною корпоративною угодою з API-провайдером.

Загальні висновки

1) Локальна й хмарна моделі використовують єдиний інтерфейс, тому перехід між ними не потребує змін у логіці програми – це основа стратегії резервування провайдерів у реальних проєктах;

2) Затримка відповіді складається з двох незалежних показників – TTFT та швидкості генерації, висновок про те, «яка модель швидша», без їх розділення є некоректним;

3) Режим міркування є інструментом із чіткою областю застосування: він окупається на багатокроковому виведенні й є чистою перевитратою на фактичних питаннях і переказі;

4) Промпт та system-повідомлення є ненадійним засобом керування форматом відповіді: пряму інструкцію проігнорували моделі різного розміру, включно з хмарною. Надійність досягається програмною валідацією, а не формулюванням;

Список використаних джерел

Lumière Brothers First Commercial Film Screening (1895):

The FilmCraft / Historical Archives. "On December 28, 1895, the world's first commercial movie screening took place at the Grand Café in Paris by Louis and Auguste Lumière."

URL: https://thefilmcraftproductions.com/the-first-commercial-movie-ever-screened/

Groq LPU (Language Processing Unit) Architecture:

Groq Technical Papers & NVIDIA LPX Architecture Documentation. "Groq LPU design utilizes high-bandwidth SRAM (up to 80–150 TB/s) to overcome memory-bandwidth bottlenecks during LLM inference."

URL: https://groq.com/blog/the-groq-lpu-explained

Chain-of-Thought Prompting in Large Language Models:

Wei et al. (2022). "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." NeurIPS / OpenReview.

URL: https://openreview.net/pdf?id=_VjQlMeSB_J

Ollama & OpenAI REST API Specification:

OpenAI Developer Documentation. "Standardization of /v1/chat/completions endpoint for portable LLM routing."